

電腦圖學 Homework #1

A Drawing Panel

黃俊源 01257128 2026-10-06

Contents

1 專案介紹	1	2.4 階層式選單	3	3.5 Command History 與 Undo/Redo	7
1.1 作業目標	1	2.5 快捷鍵與滑鼠操作	4	3.6 Rendering Pipeline . .	8
1.2 系統概述	1	2.6 視窗與對話框	4	4 繪圖與渲染實作	9
2 使用者操作說明	2	3 系統架構與設計	5	4.1 圖形的幾何表示	9
2.1 主畫面	2	3.1 整體架構	5	4.2 互動式繪圖與 Draft . . .	9
2.2 繪圖工具	2	3.2 Application 與 Window	6	4.3 Stroke 繪製	10
2.3 繪圖屬性	2	3.3 Document、Scene 與 Scene Object	6	4.4 Fill 繪製	14
		3.4 Event 與 Input	7	4.5 物件選取與 Hit Test . . .	14
				4.6 座標系統與 Grid 渲染 . .	15

1 專案介紹

1.1 作業目標

本次作業的目標是使用 OpenGL 與 GLUT 實作一套互動式 2D Drawing Panel。使用者需要能夠透過滑鼠與鍵盤在視窗中繪製基本幾何圖形與文字，並利用階層式選單選擇繪圖工具及調整顏色、線寬、點大小、填滿模式與字型等屬性。

除了基本繪圖功能外，作業也要求程式處理 keyboard、mouse、window 與 motion event，使使用者可以透過滑鼠操作決定圖形的位置與形狀。在進階要求中，程式還需要利用 idle event 或 timer 保存 Color Buffer，並在 display event 發生時還原已保存的畫面內容；此外也包含儲存 Color Buffer、輸入文字，以及利用 Grid 或 Scale 顯示位置與尺寸等功能。

1.2 系統概述

本次實作的程式命名為 **Graphtoria**，是一套以 OpenGL 與 FreeGLUT 實作的 2D 繪圖程式。

程式目前提供 Select、Point、Pencil、Line、Rectangle、Ellipse、Polygon 與 Text 等工具。使用者可以利用滑鼠直接在 Canvas 上建立與選取物件，並透過右鍵階層式選單調整 Stroke、Fill、Point Size、Text Font 與 Grid 等繪圖屬性。

除了基本繪圖功能外，程式也支援 Undo/Redo、物件刪除、多視窗、Grid、文字輸入對話框，以及 PPM 圖片輸出。Select 工具可以選取畫面中最上層的圖形或文字，並顯示目前物件的選取範圍；刪除操作也會加入 Undo/Redo 紀錄，因此可以復原。

在文字方面，除了 FreeGLUT 內建字型外，程式也支援自訂的點陣字型與 UTF-8 中文字元顯示。畫面內容則可以透過 Color Buffer 保存與還原，並可將目前 Canvas 輸出為 PPM 圖片。

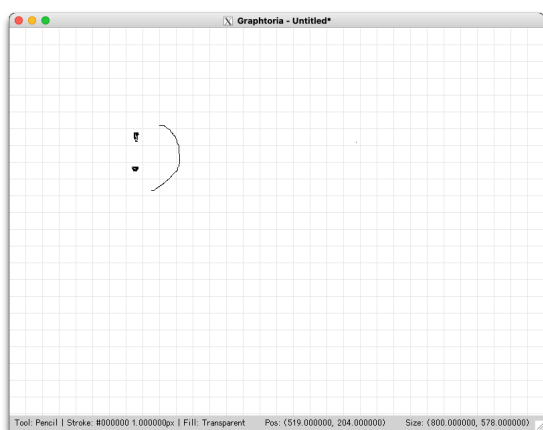
整體而言，程式除了完成作業要求的 drawing panel 功能外，也加入文件操作、物件選取與較完整的視窗互動功能。各功能的使用方式將於下一章介紹，程式內部的模組分工與資料流程則於第 3 章說明。

2 使用者操作說明

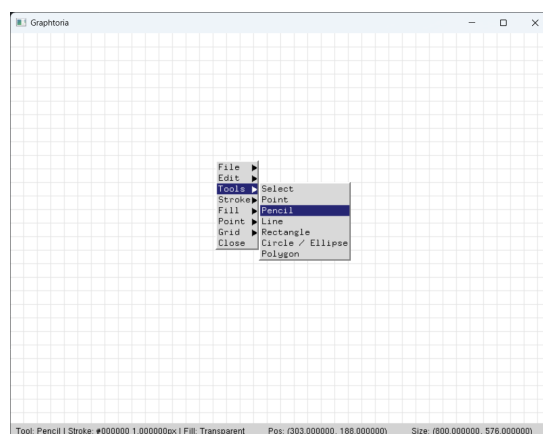
2.1 主畫面

程式啟動後會建立主要繪圖視窗，如圖 1a 所示。視窗中央為 Canvas，使用者可以在其中繪製圖形、輸入文字或選取既有物件；底部的狀態列則顯示目前使用的工具、繪圖樣式、滑鼠在 Canvas 中的位置，以及 Canvas 的尺寸。預設繪圖工具為 Pencil，Grid 預設使用 Lines 模式。

在 Canvas 上按下滑鼠右鍵可以開啟階層式選單，用來切換繪圖工具及調整 Stroke、Fill、Point、Text 與 Grid 等屬性。圖 1b 展示了主畫面與階層式選單的實際操作畫面。



(a) 繪圖程式主畫面



(b) 主畫面與階層式選單

Figure 1: 程式主畫面與階層式選單

2.2 繪圖工具

目前提供 Select、Point、Pencil、Line、Rectangle、Ellipse、Polygon 與 Text 等工具，各工具的主要操作方式整理於表 1。

2.3 繪圖屬性

程式可以分別設定圖形的 Stroke、Fill、Point 與 Text 屬性。

Stroke 可調整顏色、線寬，以及 MITER、BEVEL、ROUND 三種 Line Join；線段端點則提供 BUTT、SQUARE、ROUND 三種 Line Cap。Fill 支援 OUTLINE、FILLED 與 ADVANCED 三種模式，其中 ADVANCED 會分別處理填色與自製的粗線幾何。

Point Size 與 Stroke Width 可以透過右鍵選單調整，也可以利用中括號快捷鍵快速增加或減少大小。文字則可以選擇 FreeGLUT 內建字型或自製的點陣字型。

Table 1: 繪圖工具與操作方式

工具	操作方式
Select	單擊物件進行選取；Delete 或 Backspace 可刪除目前選取的物件，Esc 或點擊空白處可取消選取。
Point	在 Canvas 上單擊建立一個點，點的大小由目前的 Point Size 決定。
Pencil	按住滑鼠左鍵拖曳即可自由繪製，放開滑鼠後完成目前的 Path。
Line	按住滑鼠左鍵拖曳決定線段的起點與終點；繪製時按住 Shift 可限制為水平、垂直或 45° 方向。
Rectangle	按住滑鼠左鍵拖曳決定矩形的兩個對角；按住 Shift 可限制為正方形。
Ellipse	按住滑鼠左鍵拖曳決定橢圓的外接矩形；按住 Shift 可限制為正圓。
Polygon	單擊滑鼠左鍵依序加入頂點，雙擊或按下 Enter 完成圖形。Backspace 可移除上一個頂點，Esc 則取消本次繪製。
Text	在欲放置文字的位置單擊後開啟文字輸入對話框，輸入完成後將文字加入 Canvas。

2.4 階層式選單

在 Canvas 上按下滑鼠右鍵即可開啟主選單。選單依功能分成數個子選單：

- **File**：建立文件、開啟文件、儲存及輸出圖片。
- **Edit**：Undo、Redo 與清空目前畫布。
- **Tools**：切換目前使用的繪圖或選取工具。
- **Stroke**：設定線條顏色、寬度、Join 與 Cap。
- **Fill**：設定填滿模式與填色。
- **Point**：調整 Point Size。
- **Text**：選擇文字字型與文字顏色。
- **Grid**：切換格線的顯示模式。

常用功能也提供鍵盤快捷鍵，因此不需要每次都透過選單操作。

2.5 快捷鍵與滑鼠操作

程式內部以 Primary 表示各作業系統慣用的主要修飾鍵¹。例如表 2 中的 Primary+S，在 macOS 為 Command+S，在其他平台則為 Ctrl+S。

Table 2: 程式快捷鍵

快捷鍵	功能
Primary+Z	Undo；若目前仍有未完成的 draft，則先取消此次操作。
Primary+Shift+Z	Redo。
Primary+N	建立新文件；若目前內容尚未儲存，會先顯示確認視窗。
Primary+Shift+N	建立新的繪圖視窗。
Primary+O	開啟文件。
Primary+S	儲存目前文件。
Primary+Shift+S	另存新檔。
Primary+E	將目前 Canvas 輸出為 PPM P6 圖片。
Primary+R	重新整理目前視窗，並重設鍵盤與滑鼠輸入狀態。
C	清空目前畫布。
G	依序切換 Lines、Dots 與 None 三種 Grid 模式。
0	選擇 Select。
1	選擇 Pencil。
2	選擇 Line。
3	選擇 Rectangle。
4	選擇 Ellipse。
5	選擇 Polygon。
6	選擇 Text。
[	Stroke Width 與 Point Size 減少 1 px，最小為 1 px。
]	Stroke Width 與 Point Size 增加 1 px。
Shift+[	Stroke Width 與 Point Size 減少 5 px，最小為 1 px。
Shift+]	Stroke Width 與 Point Size 增加 5 px。

滑鼠左鍵主要用於繪圖與物件選取，右鍵則用來開啟階層式選單。各繪圖工具的詳細滑鼠操作方式見表 1。

2.6 視窗與對話框

程式支援同時建立多個繪圖視窗。每個繪圖視窗都有各自獨立的 Document 與 Undo/Redo 紀錄，因此不同視窗中的繪圖內容與操作歷史不會互相影響。

部分操作會另外開啟對話框。例如，Text 工具會使用文字輸入對話框取得使用者

¹macOS 對應 Command，Windows 與 Linux 對應 Ctrl。

輸入的內容；執行可能捨棄目前文件內容的操作時，則會使用確認視窗詢問是否繼續。當 Modal Dialog 開啟期間，其他繪圖視窗會暫停接收使用者輸入，避免在對話框尚未處理完成時誤操作背景視窗。

若目前文件仍有尚未儲存的修改，執行 New、Load 或 Close 等操作時，程式會先顯示確認視窗，再由使用者決定是否繼續。

圖 2 顯示目前程式使用的兩種主要對話框：左側為未儲存文件的確認視窗，右側為 Text 工具使用的文字輸入對話框。



Figure 2: 程式中的確認視窗與文字輸入對話框

3 系統架構與設計

3.1 整體架構

為了避免繪圖、事件處理與 OpenGL 呼叫全部集中在 GLUT callback 中，程式將主要功能拆分為視窗管理、文件資料、繪圖工具與畫面渲染等部分，整體架構如圖 3 所示。

Application 負責建立與管理各個視窗，PaintWindow 則組合繪圖介面、選單、快捷鍵與目前的 Document。使用者在 Canvas 上完成一次操作後，產生的繪圖物件會交由 Document 修改 Scene；顯示時再由 Renderer 讀取 Scene 中的內容並繪製到畫面上。

這樣的分工讓使用者操作、文件資料與渲染流程彼此分開。例如 Undo/Redo 只需要處理文件內容的修改，Renderer 也不需要知道物件是透過哪一種工具建立的。



Figure 3: 繪圖程式的整體架構

3.2 Application 與 Window

Application 負責 FreeGLUT 的初始化，以及所有視窗的建立與生命週期管理。除了主要的 PaintWindow 外，文字輸入、確認視窗與圖片輸出所使用的視窗也共用相同的管理方式。各視窗類別的繼承關係與 Application 的管理關係如圖 4 所示。

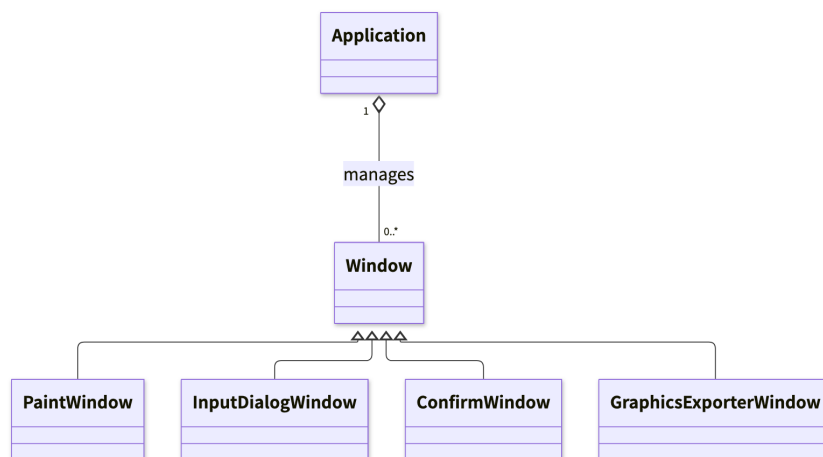


Figure 4: Application 與視窗類別關係

FreeGLUT 使用 C-style callback，因此無法直接將 callback 綁定到某一個 C++ 物件²。為了解決這個問題，每個 Window 建立時都會記錄自己的 GLUT window ID；callback 發生後，再利用目前的 window ID 找回對應的 Window instance，最後將輸入交給該視窗處理。

Menu 的處理方式也類似。FreeGLUT 的 menu callback 只提供整數 ID，程式再透過這個 ID 找回對應的選單項目並執行其動作。

較完整的 callback routing、Modal Dialog 與多視窗生命週期整理於附錄 ??。

3.3 Document、Scene 與 Scene Object

Document 代表一份繪圖文件，其中保存檔案資訊、Canvas 尺寸、Scene 與 CommandHistory。新增、刪除或清除繪圖內容時，都會透過 Document 修改，而不是由繪圖工具直接操作 Scene。

Scene 則依照繪製順序保存所有繪圖物件。目前主要分為 ShapeObject 與 TextObject：前者用來表示 Point、Line、Rectangle、Ellipse、Path 與 Polygon，後者則保存文字位置、UTF-8 字串與文字樣式。

繪圖工具在操作完成後只需要產生對應的 Scene Object，再交由 Document 建立 Command 並加入 Scene。因此 Scene 本身只負責保存資料，不需要知道 UI、工具或

²Non-static member function 具有隱含的 **this** 參數，其函式型別與 FreeGLUT 所要求的 callback signature 不相同。

OpenGL 的存在。

拖曳期間顯示的 **draft** 與 **Select** 的選取框則屬於暫時的互動狀態，不會加入 **Scene**。只有當繪圖操作真正完成，或使用者確定刪除物件時，才會產生對文件內容的修改，因此也不會在操作過程中產生多餘的 **Undo** 紀錄。

圖 5 整理了 **PaintWindow**、**Document** 與場景物件之間的擁有與繼承關係。

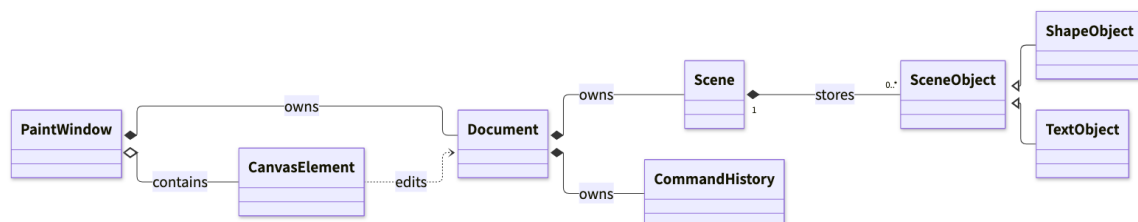


Figure 5: Document、Scene 與 Scene Object 類別關係

3.4 Event 與 Input

FreeGLUT 提供的 **keyboard**、**mouse** 與 **motion callback** 會先由 **Window** 轉換成程式內部使用的 **Event**。**Event** 中除了事件本身的資料外，也包含目前的 **keyboard** 與 **mouse state**，讓後續元件可以用一致的方式判斷滑鼠位置、按鍵與 **Shift** 等狀態。

滑鼠事件會先經過 **hit test** 找出對應的 **Element**。若目前已有 **Element** 取得 **mouse capture**，則事件會優先交給該 **Element**，使拖曳操作即使暫時離開原本區域仍能正常收到 **MouseUp**。

因此工具本身不需要直接處理 GLUT 的原始 **callback**，只需要回應整理過後的 **Event**。**Event bubbling**、**focus**、**mouse capture** 與 **shortcut matching** 等細節則收錄於附錄 ??。

3.5 Command History 與 Undo/Redo

為了支援 **Undo/Redo**，所有會修改文件內容的操作都會封裝成可復原的 **Command**。目前新增、插入、移除物件與清空畫布等操作，都會經由 **CommandHistory** 執行。

執行新的 **Command** 後，該操作會加入 **undo stack**；使用 **Undo** 時會復原這次修改，並將 **Command** 移到 **redo stack**；**Redo** 則再次執行同一筆 **Command**。若 **Undo** 後又執行新的操作，原本的 **redo stack** 便會被清除。

每筆紀錄另外保存操作前後的状态 ID，用來判斷目前文件是否仍與最近一次儲存的狀態相同。移除物件時也會記錄它原本在 **Scene** 中的位置，因此 **Undo** 後可以恢復原來的前後層級。

圖 6 整理了 Command History 的主要流程。

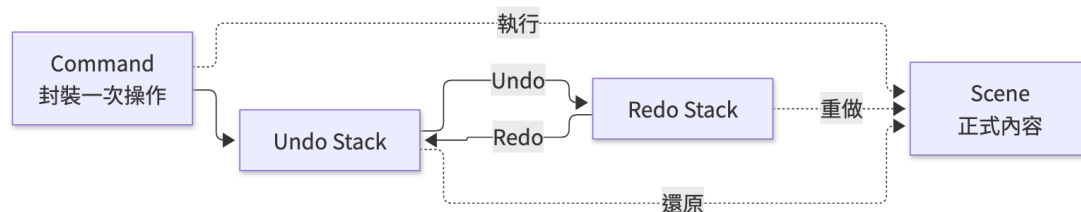


Figure 6: Command、Undo 與 Redo 的資料流

3.6 Rendering Pipeline

當視窗需要重新顯示時，Window 會從 Root Element 開始走訪 Element tree，並讓各個 Element 在自己的局部座標系中進行繪製。當流程進入 Canvas 時，後續的繪圖工作會交由 CanvasRenderer 處理。

Canvas 內的內容具有固定的前後順序。最先繪製的是背景 Grid，接著按照 Scene 中的物件順序繪製已完成的圖形與文字；工具尚未完成的 draft 會疊在正式內容上方，最後才繪製 Select 工具的選取框。因此畫面由底到頂的合成順序為：

Grid → Scene Objects → Tool Draft → Selection Overlay.

對於 Scene 中的正式物件，SceneRenderer 會依照物件類型將繪製工作分派給不同的 Renderer。ShapeRenderer 負責幾何圖形的 Stroke 與 Fill，而 TextRenderer 則負責文字與 Glyph 的繪製，最後再轉換成實際的 OpenGL 繪圖命令。

這樣的設計將「畫面內容的前後層級」與「不同物件的繪製方式」分開處理：Canvas 決定各類內容的合成順序，而各 Renderer 則只負責如何將指定物件畫到畫面上。整體流程如圖 7 所示。

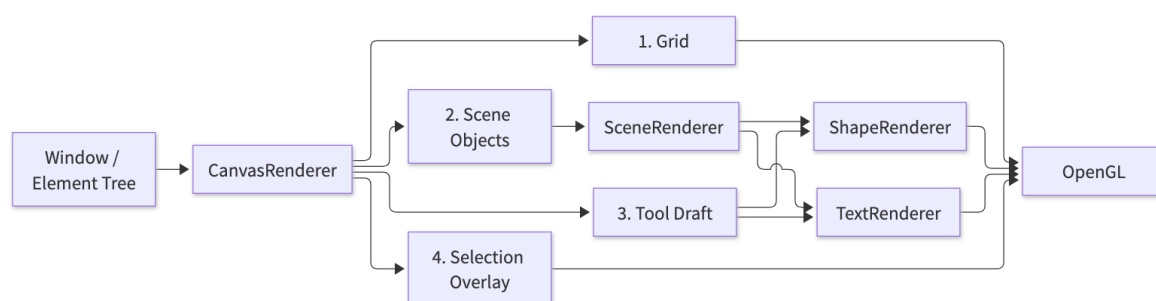


Figure 7: 畫面渲染流程

4 繪圖與渲染實作

4.1 圖形的幾何表示

不同繪圖工具取得圖形的方式並不相同。例如 **Line** 與 **Rectangle** 由滑鼠拖曳決定範圍，**Pencil** 記錄滑鼠移動形成 **Path**，**Polygon** 則由使用者逐點建立邊界。這些差異主要存在於互動階段；當繪圖完成後，圖形會整理成 **Renderer** 可以直接使用的幾何資料，因此後續的繪製不需要再區分頂點原本是如何輸入的。

程式中的 **Line**、**Rectangle**、**Path** 與 **Polygon** 最後都可以表示為一組頂點，並記錄圖形是否形成封閉邊界。**Ellipse** 則依照外接矩形計算中心與半徑，將橢圓取樣為 64 個頂點後，以相同的方式交給 **Renderer** 處理。**Point** 較為單純，只需要保存位置，繪製時再配合目前的 **Point Size**。

這樣的表示方式也成為後續繪圖處理的共同基礎。**Renderer** 可以由相同的頂點資料建立 **Stroke** 與 **Fill**，**Select Tool** 也可以利用圖形的幾何資訊進行 **Hit Test**。至於使用者在拖曳或連續輸入頂點時，如何從暫時的預覽形成正式物件，則於下一節說明。

4.2 互動式繪圖與 Draft

繪圖工具取得的幾何資料通常不是一次完成的。例如 **Line**、**Rectangle** 與 **Ellipse** 需要由滑鼠拖曳決定範圍，**Pencil** 會在拖曳期間持續收集路徑，**Polygon** 則需要經過多次點擊才能形成完整圖形。在這些操作真正完成以前，畫面仍需要即時顯示目前的繪製結果。

因此，程式將互動過程中的圖形保存為暫時的 **Draft**。使用者開始繪圖後，後續的滑鼠移動只會更新 **Draft**，**Canvas** 再將它繪製於既有 **Scene** 之上作為預覽；此時並不修改正式的文件內容。直到使用者完成操作後，**Draft** 才會轉換成正式的繪圖物件，再經由 **Document** 加入 **Scene**。整體關係如圖 8 所示。

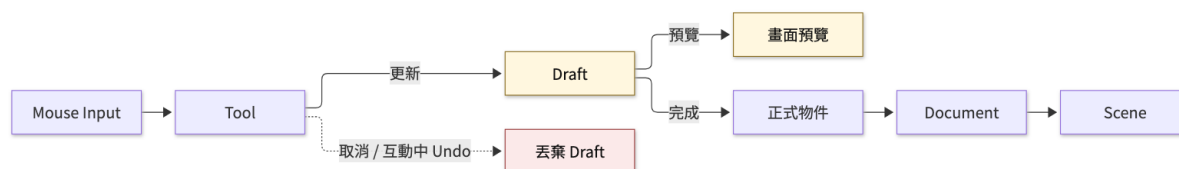


Figure 8: 互動式繪圖中 Draft 與正式物件的關係

這樣可以將「正在進行的互動」與「已完成的文件內容」分開。拖曳過程即使產生大量 **Motion event**，也只是在更新同一個 **Draft**，不會為每次移動建立一筆修改紀錄。因此一次完整的繪圖操作只會形成一次可復原的文件修改。若操作尚未完成就執行 **Undo**，程式也會優先取消目前的 **Draft**，而不是復原 **Scene** 中前一個已完成的物件。

4.2.1 Pencil 的路徑採樣

Pencil 與其他拖曳工具稍有不同。Line 或 Rectangle 只需要更新目前的終點，Pencil 則必須在滑鼠移動過程中持續加入新的頂點，才能保存使用者實際畫出的軌跡。然而，程式並不會將每一次 Mouse Motion 都直接加入 Path。

設上一個採樣點為 P_{i-1} ，目前的滑鼠位置為 P ，Stroke Width 為 w 。程式使用的最小採樣距離為

$$d_{\min} = \max(0.2w, 1 \text{ px}).$$

只有當

$$\|P - P_{i-1}\| \geq d_{\min}$$

時，才會將目前位置加入 Path。因此 Stroke Width 增加時，Pencil 的最小採樣間距也會隨之增加。

這項限制除了避免在很短的距離內產生大量幾乎重合的頂點，也能減少後續 Stroke 計算遇到極短線段的情況。Stroke Geometry 需要由相鄰頂點計算線段方向與轉角；當兩個頂點距離過近時，微小的座標差異便可能造成較大的方向變化，進一步使轉角的幾何計算不穩定。因此 Pencil 在輸入階段先以距離門檻控制採樣，再將取得的頂點交給後續的 Stroke Renderer 處理。

4.3 Stroke 繪製

基本模式直接使用 OpenGL 內建的線條繪製功能，並透過 `glLineWidth()` 設定線寬。這種方式實作簡單，但在繪製由多條線段組成的粗折線時，轉角的外形無法由程式進一步控制。如圖 9 左側所示，斜向線段的接合處可能產生明顯的突出。

因此，ADVANCED 模式不直接使用 OpenGL 的粗線結果，而是自行建立 Stroke 的幾何區域，以控制線條的轉角與端點外形。圖 9 右側為自行處理後的結果。後續將分別說明 Stroke 區域的建立方式、Line Join、Line Cap，以及尖銳轉角的處理。

4.3.1 由中心線建立粗線區域

自行繪製粗線時，首先需要將沒有寬度的中心線轉換成實際的粗線區域。對一條由 A 指向 B 的非零長度線段，先求出單位方向 $\vec{u}$ ，再取得與其垂直的單位向量 $\vec{n}$ ：

$$\vec{u} = \frac{B - A}{\|B - A\|}, \quad \vec{n} = (-u_y, u_x).$$

若 Stroke Width 為 w ，令半寬 $h = w/2$ 。對中心線上的任一點 P ，沿 $\vec{n}$ 的正、反方向各偏移 h ，即可得到兩側邊界：

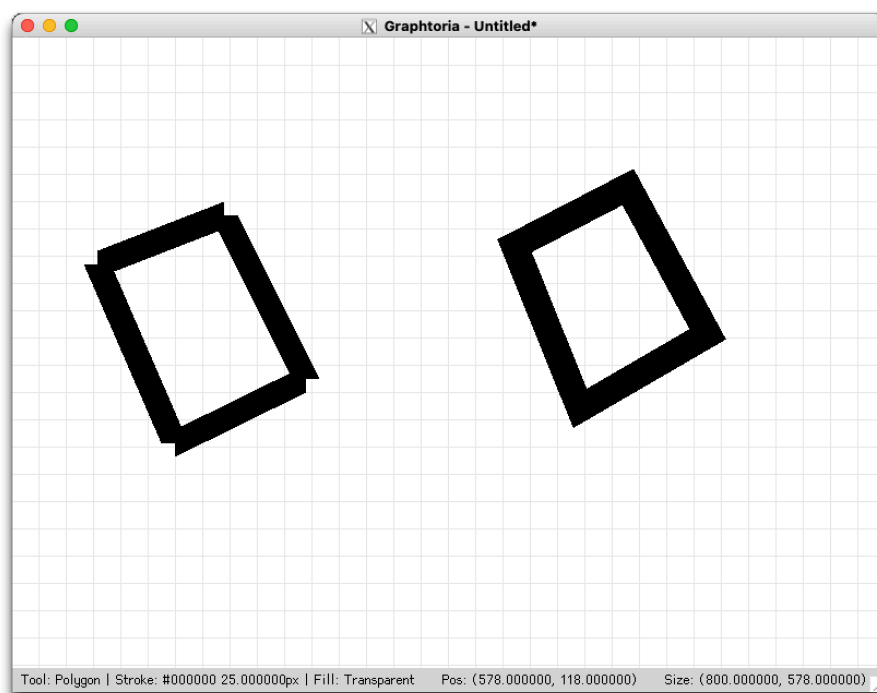


Figure 9: OpenGL 原生粗線與自行實作 Stroke 的轉角比較

$$P_L = P + h\vec{n}, \quad P_R = P - h\vec{n}.$$

因此線段的兩個端點 A 、 B 分別產生

$$\begin{cases} A_L = A + h\vec{n}, & A_R = A - h\vec{n}, \\ B_L = B + h\vec{n}, & B_R = B - h\vec{n}. \end{cases}$$

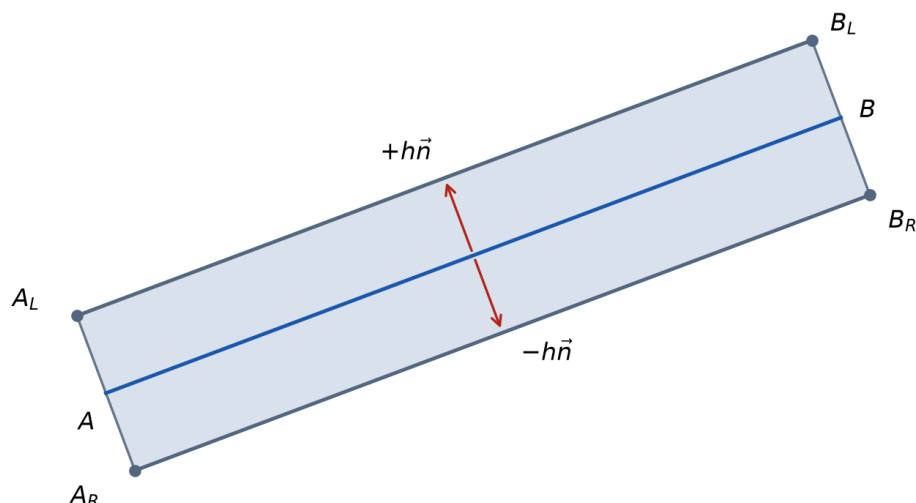
如圖 10 所示， A_L 、 B_L 、 B_R 、 A_R 四個邊界點形成一個四邊形，再拆成兩個三角形交由 OpenGL 繪製。這就是單一線段形成粗線區域的基本方式。

單一線段只需要上述四個邊界點即可完成繪製，但當多條線段共用一個頂點時，兩段粗線的邊界還需要在轉角處互相連接；開放路徑的首尾也需要決定端點的外形。這兩種情況分別由 Line Join 與 Line Cap 處理。

4.3.2 Line Join 與 Line Cap

Line Join 決定相鄰兩條線段在轉角處如何接合。本程式提供 MITER、BEVEL 與 ROUND 三種方式，其差異如圖 11a 所示。

MITER 將兩段外側邊界延伸至交點，因此能保留尖銳的轉角；BEVEL 直接連接兩個外側端點，將尖角切平；ROUND 則以原頂點為圓心建立圓弧，並以多個三角形近似圓形的轉角。

Figure 10: 中心線向兩側偏移半寬 h 形成粗線區域

Line Cap 則決定開放路徑首尾的形狀。BUTT 直接在原本的端點停止；SQUARE 沿中心線方向向外延伸半個 Stroke Width；ROUND 則以端點為圓心形成半圓。封閉路徑的首尾彼此相接，因此所有頂點皆以 Line Join 處理，不需要額外建立 Line Cap。

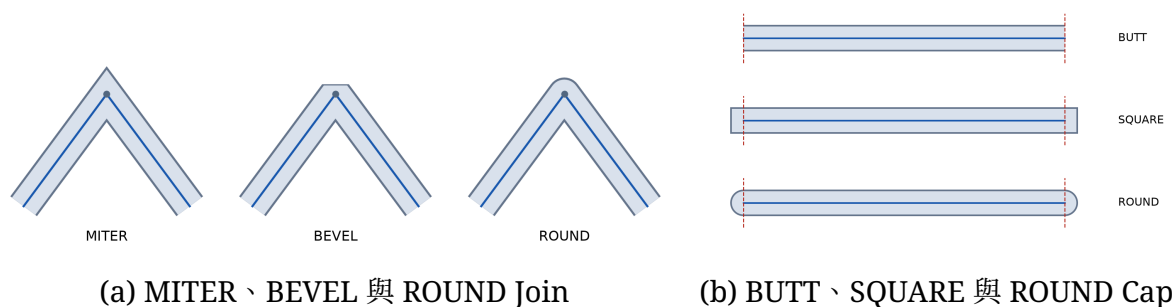


Figure 11: Line Join 與 Line Cap 的不同形式；細線表示原始中心線

4.3.3 Miter Limit

MITER 雖然能保留尖銳的轉角，但當兩條線段形成的夾角越小時，外側邊界的交點會逐漸遠離原頂點，使 Stroke 產生過長的尖角。

設兩條中心線在頂點形成的夾角為 θ ，線寬的一半為 h ，原頂點至 MITER 外側交點的距離為 m 。由圖 12 所示的幾何關係可得

$$m = \frac{h}{\sin(\theta/2)}.$$

由此可知，當夾角 θ 越小時，MITER 的尖角會越長。因此若不限制 MITER 的長度，在較尖銳的轉角處可能產生過度延伸的尖角。

為避免這種情況，程式使用 Miter Limit L 。只有在

$$m \leq hL$$

時才使用 MITER；若 m 超過限制，則改用 BEVEL。

結合前述 m 與 θ 的關係，可得

$$\theta \geq 2 \arcsin\left(\frac{1}{L}\right).$$

目前 miterLimit 預設為 4，因此允許使用 MITER 的最小夾角約為 29.0° 。圖 12 分別示範 m 位於 Miter Limit 以內而使用 MITER，以及 m 超過限制而改用 BEVEL 的情況。

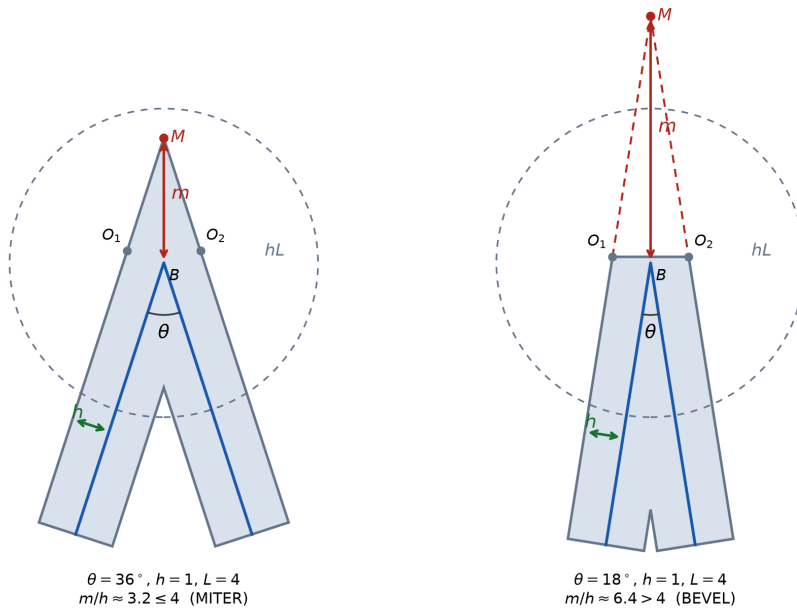


Figure 12: Miter length m 與限制長度 hL 的關係

4.3.4 退化情況處理

實際建立 Stroke 區域時，還需要考慮長度過短、近平行或接近折返的線段。當線段長度過短時，方向向量的計算容易受到微小座標差異影響；而相鄰邊界接近平行時，其交點也可能變得不穩定或距離頂點過遠。

因此程式會排除重複或退化的線段，並在無法穩定取得合理交點時採用較保守的接合方式，避免產生異常延伸的三角形。前述 Pencil 的最小距離採樣，也能減少極短線段進入 Stroke 計算的情況。

這類數值穩定性問題與 Miter Limit 所處理的尖角問題來源不同：前者主要發生在退化或接近退化的幾何情況，後者則來自 MITER 在小夾角下會產生較長尖角的特性，因此程式分別進行處理。

4.4 Fill 繪製

封閉圖形除了 Stroke 外，也可以填滿內部區域。在基本的 FILLED 模式中，程式使用 OpenGL 的 GL_FILL 繪製封閉圖形；ADVANCED 模式則將 Fill 與 Stroke 分開處理，先繪製內部填色，再使用前一節自行實作的方式繪製 Stroke。

ADVANCED 模式的 Fill 直接使用 GL_POLYGON。Renderer 依照圖形的頂點順序將所有頂點送入 OpenGL，並使用目前設定的 Fill Color 完成填色。若 Fill Color 完全透明，或圖形少於三個頂點，則略過 Fill。

```
1 glColor4fv(fillColor);
2 glBegin(GL_POLYGON);
3
4 for (const auto& v : vertices) {
5     glVertex2f(v.x(), v.y());
6 }
7
8 glEnd();
```

相較於 Stroke 需要自行計算線寬、轉角與端點，Fill 不需要額外建立邊界，因此實作較為直接。目前 Fill 仍由 OpenGL 的 GL_POLYGON 完成，程式本身沒有另外實作多邊形三角化。

4.5 物件選取與 Hit Test

Select Tool 需要根據滑鼠位置判斷使用者所指向的物件。由於不同圖形的可見區域並不相同，程式不只使用外接矩形進行判定，而是依照 Fill、Stroke 與物件種類分別處理。

對具有可見 Fill 的封閉圖形，滑鼠位於圖形內部時即可視為命中。程式使用 Point-in-Polygon 判斷測試點是否位於多邊形內部：由測試點向水平方向延伸射線，依序檢查多邊形各邊是否與射線相交，每遇到一次交點便切換一次內、外狀態。完成所有邊的檢查後，即可判斷測試點是否位於圖形內。

對 Stroke 的判定則以滑鼠位置至各線段的最短距離為基礎。設測試點為 P ，線段兩端為 A 、 B ，先將 $P - A$ 投影至線段方向，得到

$$t = \frac{(P - A) \cdot (B - A)}{\|B - A\|^2}.$$

由於要計算的是到「線段」而非無限延伸直線的距離，程式將 t 限制在 $[0, 1]$ ，再取得線段上距離 P 最近的點

$$Q = A + t(B - A).$$

最後計算

$$d = \|P - Q\|.$$

當 d 小於 Hit Test 的容許距離時，即判定 **Stroke** 被選中。容許距離會考慮目前實際的 **Stroke Width**，並額外保留少量範圍，使較細的線條不需要精確點在線條的像素上才能選取。**Point** 也採用相同概念，但直接計算滑鼠與 **Point** 位置之間的距離，並以 **Point Size** 決定其可選取範圍。

文字物件則使用不同方式處理。程式根據目前字型計算文字的寬度與高度，形成一個矩形範圍，再判斷滑鼠位置是否落在此範圍內。

當同一位置同時命中多個物件時，**Select Tool** 會由 **Scene** 的尾端向前搜尋。由於 **Scene** 中較後面的物件也較晚繪製，因此這種搜尋順序會優先選取視覺上位於上層的物件。

選取完成後，程式另外計算物件的外接矩形，並將 **Stroke Width** 或 **Point Size** 納入邊界範圍，最後在 **Scene** 與工具預覽之後繪製選取框，使目前選取的物件能在畫面上清楚辨識。

4.6 座標系統與 Grid 渲染

GLUT 的滑鼠事件以視窗左上角為原點， x 軸向右、 y 軸向下。為了讓輸入與 OpenGL 繪圖使用相同方向，程式採用左上角原點的正交投影：

```
1 glOrtho(0.0, width, height, 0.0, -1.0, 1.0);
```

因此滑鼠座標不需要另外反轉 y 軸。由於 **Canvas** 只是 **Element tree** 中的一個元素，事件進入 **CanvasElement** 後，會透過 `windowToLocal()` 轉換成以 **Canvas** 左上角為原點的局部座標。

渲染時，每個 **Element** 也會先將座標系平移到自己的左上角，因此輸入與繪圖最後都使用相同的 **Canvas** 局部座標系。

Grid 亦在此座標系中由 **CanvasRenderer** 繪製。**Lines** 模式產生等距水平與垂直線，**Dots** 模式則只繪製交點。**Grid** 會先於 **Scene** 繪製，因此只作為畫布背景，不會覆蓋正式物件、**Draft** 或選取框。